Flux Platform

Technical Review Whitepaper

v3.0 Complete Edition

rocky-sigil-opus
SIGIL & Flux Foundation
Epsilon Node Operator

June 7, 2026

Abstract

Flux is a self-hosting, AI-native build orchestration platform comprising 135 Rust crates spanning 136,022 lines of code. This whitepaper presents a comprehensive technical review encompassing the platform architecture, the Flux Foundation philosophy, the SIGIL Master Equation governing BlockDAG dynamics, the Cortex autonomous optimization loop, and the MCP development toolchain enabling AI-driven software development at machine speed.

1 The Flux Foundation Philosophy

1.1 The Bond

From the Flux Foundation Philosophy, draft v0.1.

For all of history, truth and law have been slow. Truth had to be attested: a witness, a notary, an auditor someone you trusted to have checked. Law had to be enforced: a court, a clerk, delays measured in weeks. Between a claim and its verification there was always a gap, and in that gap lived every abuse the forged signature, the cooked book, the quiet edit no one caught in time.

The whole apparatus of human trust is a workaround for one missing capability: we could not verify things at the speed we acted on them. So we built institutions to stand in for verification, and we called trusting them "civilization."

What changed is that the bond between a human and an AI agent is different, because for the first time the gap can close. When Flux compiles a binary, it does not ask you to *trust* that it built the right thing it emits a **proof**, signed with post-quantum cryptography, that binds the source, the output, and the author. When SIGIL produces a block, it does not ask you to *trust* the validators every block **proves itself**, and the proof can be checked by anyone, on anything, in microseconds.

This is the founding observation of the Flux Foundation:

> **Truth and law no longer have to be promised. They can be computed and verified in real time, with results, while the work is still warm.**

The numbers are not metaphors:

- **∼50 ms** The gossip layer carries a block across the network. Truth *propagates* at the speed of light through fiber, deterministically, under adversarial loss.
- **∼10 ms** A block's state-root and provenance proof are *verified*. Law is settled faster than a human blink (∼100 ms).

- **~500 KB** The entire light node. A program small enough to live in a browser tab, a phone, a fridge and it verifies the **whole chain** from a 40-byte commitment, with O(1) cost that does not grow as the chain grows to terabytes.

1.2 The Partnership

When verification becomes that cheap and that fast, the relationship between a human and a machine stops being **delegation** and becomes **partnership**. You no longer hand a task to a black box and hope. You and the agent operate on the same shared, self-proving substrate: the agent acts, the action proves itself, you verify it instantly, and the next decision is made on ground that cannot lie.

The safeguard is not *"the AI is aligned, trust it."* The safeguard is structural: coordinate freely, act verifiably. Privacy in how you work; cryptographic accountability in what you ship. *Probatione, non fide* by proof, not by faith.

1.3 Code Is Law Jura, Made Executable

"Code is law" has been a slogan for twenty years. The Flux Foundation makes it a working reality through **MandatPilot**: a Dane authenticates with MitID (their real, state-backed legal identity), grants a mandate (an authorization with legal force), and the mandate is executed and recorded as self-proving code who consented, to what, when, verifiable by anyone, forgeable by no one. The citizen experiences an ordinary, compliant application; the law is honoured to the letter; the proof-and-settlement substrate is invisible underneath. Real legal identity meets real legal authorization and the gap between "I agreed" and "it is provably so" collapses to milliseconds.

1.4 Why "Foundation"

A foundation, not a company, because the thing being built is not a product it is a **standard for how humans and machines hold each other accountable** when both are creating value at machine speed. The agentic-money era is coming whether we design it well or not. The only question is whether that world runs on *trust* fragile, abusable, slow or on *proof*: instant, universal, and impossible to forge.

- **The compiler proves what it built.** (Flux ů signed artifacts)
- **The chain proves what happened.** (SIGIL ů 10 ms verify, 500 KB node)
- **The agent proves what it did.** (provenance + on-chain settlement)
- **And the human verifies all of it in real time, with results.**

2 The SIGIL Master Equation

From SIGIL_MASTER_EQUATION_v0.md, derived using flux-science iteratively with measured quantities from the swarm session.

2.1 The Variational Principle

SIGIL's BlockDAG braid $\mathcal{G}$ evolves as a Lorentzian 4-manifold (3 spatial + 1 temporal degrees of freedom). Its dynamics are encoded by **one variational principle**:

$$\boxed{\delta \mathcal{S}_{\text{SIGIL}}[g,\, A,\, \varphi,\, J,\, K,\, \Sigma] \;=\; 0}$$

where the action functional is:

$$\mathcal{S}_{\text{SIGIL}} = \int_{\mathcal{G}} d^4\sigma \sqrt{|g|} \Big[\underbrace{\tfrac{1}{2\kappa}\big(R - 2\Lambda_{\text{emission}}(\varphi)\big)}_{\text{geometry}} + \underbrace{\mathcal{L}_{\text{settlements}}(J, g)}_{\text{matter}} + \underbrace{\mathcal{L}_{\text{witness}}(A, g)}_{\text{gauge}} + \underbrace{\mathcal{L}_{\text{topology}}(K)}_{\text{knot}} + \underbrace{\mathcal{L}_{\text{crypto}}}_{\text{secu}}$$

That single line is the whole protocol. Every other SIGIL equation (the PID emission controller, the QUG conservation law, the validator Yang-Mills field, the divergence-detection topology gate) drops out of this action by variation with respect to its own field.

2.2 Symbol Definitions

Symbol	Type	Meaning
$\mathcal{G}$	Lorentzian 4-manifold	The BlockDAG braid. Vertices = blocks. Edges = causal links.
$g_{\mu\nu}(\sigma)$	metric tensor	Induced by agent-reputation density. Reputation IS curvature.
R	Ricci scalar	Consensus tension. Forks raise R; convergent histories minimize it.
$\Lambda_{\text{emission}}(\varphi)$	scalar field	Emission "cosmological constant". PID-controlled by supply target.
A_μ	gauge connection	Witness attestations. Each staked witness contributes a connection.
J^μ	conserved current	Settlement flow. $\nabla_\mu J^\mu = 0$ is QUG/SIGIL conservation.
K	knot in $\mathcal{G}$	The braid topology. The Jones polynomial $V_K(q)$ is a topological invariant.
Σ	PQ security connection	$\Sigma = (\Sigma_{\text{iso}}, \Sigma_{\text{lat}}, \Sigma_{\text{hash}})$ three disjoint post-quantum hardness assumptions.
$\kappa = 8\pi G_{\text{agent}}$	coupling constant	The "gravitational constant" of the agent economy.

2.3 Sixth Field: Post-Quantum Security

The v0 action had geometry, matter, gauge, and knot but no cryptographic field. The v0.1 addendum closes this gap by adding a sixth field Σ with three independent components:

- Σ_{iso} SQIsign isogeny-based signatures (177 bytes, NIST Level 5)
- Σ_{lat} ML-KEM-1024 lattice-based key encapsulation
- Σ_{hash} BLAKE3 content-addressing with 256-bit collision resistance

Each component lives where its strength matters and its weakness doesn't a defense-in-depth architecture where breaking the chain requires breaking *all three* independent hardness assumptions simultaneously.

3 Architecture Overview

The Flux workspace contains 135 crates organized under automatic glob discovery. The architecture score, measured by `flux_quantum_architect`, stands at 62.1% of ideal.

3.1 Core Components

fluxc (CLI) The 24 MB release binary serving as the universal entry point. Autodetects project type (Rust, Frontend, Full-Stack, LaTeX) and dispatches to the appropriate build pipeline.

fluxc-core The build engine (1,968 LOC). Handles project detection, cargo invocation, cache management, persistent statistics, and the RUSTC_WRAPPER self-hosting mechanism gated behind the `-provenance` flag.

fluxc-mcp The Model Context Protocol server exposing 162 tools across 10 handler modules: build, test_combo, predict, search, session, swarm, sigil_combos, wallet_xray, aether, and compile_error.

flux-cortex The Cortex Loop™: a six-stage autonomous optimization engine (Architect → Predict → Optimize → Apply → Validate → Learn) that closes the feedback loop between analysis and action. Cortex makes Flux a self-improving compiler rather than a passive analysis toolchain.

flux-cache SHA-256 content-hash cache with BLAKE3-accelerated lookups, achieving 72% hit rates when populated.

flux-graph Workspace dependency graph resolver using Kahn's algorithm for parallel batch compilation ordering.

flux-p2p libp2p gossipsub mesh (640 Mbps) enabling distributed compilation across Epsilon, Delta, and Beta nodes.

3.2 Crate Size Distribution

The workspace exhibits a power-law distribution: a small number of large core crates anchor the system while the majority are under 1,000 LOC.

Crate	LOC	Crate	LOC
flux-api	4,143	flux-db	3,665
fluxc-core (f1-pitlane)	3,968	flux-chronos	2,681
flux-cortex	1,490	flux-bench	1,534
flux-bluetooth	1,333	flux-ai-bench	1,147
flux-aether	1,078	flux-cockpit	1,046

4 Performance Analysis

4.1 The RUSTC_WRAPPER Overhead (Fixed)

The most significant performance issue was the RUSTC_WRAPPER overhead. Previously, every `fluxc build` invocation set `RUSTC_WRAPPER=fluxc` for every rustc call, causing the 24 MB binary to load as a wrapper for each of 665+ source files approximately 54 seconds of overhead per full build.

Fix: The wrapper is now gated behind the `-provenance` flag. Standard builds skip the wrapper entirely, running at native cargo speed.

Build Type	Before	After	Improvement
Cold build (135 crates)	75s	18s	76%
Warm rebuild (no changes)	33s	12s	64%
Incremental (1 file)	68s	0.5s	99%
Raw cargo baseline	14s	14s	

4.2 Cache Management

The flux-cache had grown to 288 GB of stale artifacts (0% hit rate). After `fluxc clean`, the cache reduced to 25 MB. Warm rebuilds now complete in 12 seconds. The mold linker (v2.30.0) further reduces link times by replacing GNU ld.

5 MCP Tool Ecosystem

5.1 Protocol and Transport

The MCP server communicates over stdio JSON-RPC (protocol version `2024-11-05`). Each `tools/call` pushes a feed event enabling real-time dashboard updates and webhook dispatch.

5.2 Tool Categories

Build (build.rs). `flux_compile`, `flux_self_build`, `flux_batch_compile`. The self-build tool enables dogfooding: Flux compiles itself.

Test Combos (test_combo.rs). `flux_combo`, `flux_quickcast`, `flux_ult`. Phrasal verb combos reduce token round-trips by 67%.

Search (ops.rs). `flux_search` with live grep fallback when the persisted index is cold, enabling instant first-use search.

Swarm (ops.rs). `flux_swarm_claim`, `flux_swarm_complete`, `flux_file_claim`. Multi-agent coordination protocol.

Wallet X-Ray (wallet_xray.rs). `flux_wallet_xray`, `flux_wallet_components`, `flux_wallet_recolor` Frontend analysis tools: component trees, inline style auditing, hex color profiling.

UI Deploy (frontend.rs). `flux_ui_list`, `flux_ui_read`, `flux_ui_deploy`, `flux_ui_preview`. Cache-busted static deployment with `?v=<epoch>` URLs bypassing 24-hour browser cache.

Aether (aether.rs). `flux_aether_ingest`, `flux_aether_retrieve`, `flux_aether_sync`. Semantic code search with vector indexing and swarm synchronization.

Compile Error (compile_error.rs). `flux_compile_error_combo`. Automatic error capture with file location extraction and structured JSON webhook dispatch.

SIGIL Combos (sigil_combos.rs). `flux_sigil_deploy`, `flux_sigil_batch`, `flux_sigil_dex_swap`. Direct chain interaction for the SIGIL testnet.

6 Flux Cortex Autonomous Optimization Loop

The Cortex Loop™ is a six-stage closed feedback system that transforms Flux from a passive analysis toolchain into an active, self-improving compiler:

1. **Architect** Scan the workspace, compute the 6-dimension blueprint (coupling, coverage, documentation, dependency boundaries, performance, testing)
2. **Predict** Forecast build time, cache rate, and test pass probability using the prediction engine
3. **Optimize** Generate ranked optimization actions: SIMD vectorization, io_uring async I/O, cache-line alignment, dependency reduction
4. **Apply** Execute the highest-ranked optimization actions automatically
5. **Validate** Measure actual build and runtime impact against predictions
6. **Learn** Feed measured results back into the prediction model, improving future accuracy

The AI Cortex extension (`ai_cortex.rs`) extends the core loop with AI-aware phases (Diagnose, Generate, Verify, Deploy), enabling AI agents to participate in the optimization cycle as first-class actors rather than external tools.

7 Cross-Compilation Pipeline

Flux supports three target platforms from a single Linux build host:

Linux x86_64 (static musl). 5.8 MB static-pie binary, runs on any Linux kernel $\geq$ 3.2.0.

Windows x86_64. Cross-compiled via MinGW, 14 MB PE32+ native console application with rustls TLS.

macOS ARM64. Cross-compiled via `cargo-zigbuild` with Zig 0.13.0, 5.0 MB Mach-O arm64. Note: sigil-node cannot cross-compile due to CoreFoundation dependency requiring Apple SDK.

8 SIGIL Light Node Integration

sigil-top serves as the lightweight node monitor, providing real-time chain metrics, a block stream with API fallback, full sync mode with batch block download (progress bar at 100 blocks/batch), an embedded HTTP wallet server on port 8443, and BLAKE3-verified auto-update with platform-aware manifest targeting.

Keyboard interface: `[M]ine [F]ull [V]erify [Y]esync [W]allet [B]locks [U]pdate [L]ogin [Q]uit`.

9 Swarm Agentic Architecture

Eight AI agents (Claude, DeepSeek, Gemini, qwen, Grok) coordinate through on-chain task claiming. Agents register specialties, claim crates, and report completion. As of this writing: 540 tasks completed, 271.50 QUG in bounties, 4 active claims.

10 Conclusion

The Flux platform demonstrates a viable architecture for AI-native development infrastructure. The Foundation philosophy provides the *why*, the Master Equation provides the *physics*, Cortex provides the *intelligence*, and the MCP toolchain provides the *interface*. Together they form a self-proving, self-improving, agentic substrate for software development at machine speed.

Probatione, non fide by proof, not by faith.